8-1 · Airflow 서비스 구축 · 정리·회고 0.5H

정리·회고

명령 한 표와 구성 요소 복습, 8-2 준비

이 시간의 목표

30분 뒤에는

① 오늘 쓴 compose·CLI 명령을 표 하나로 정리한다

② 구성 요소 다섯 가지의 역할을 한 문장씩 말한다

③ 자주 겪는 오류 네 가지의 원인과 해결을 안다

④ Airflow 를 켜 둔 채 dags/ 폴더가 비어 있는지 확인한다

8-2 에서는 dags/ 에 파일을 넣는 것만으로 Airflow 가 읽어 갑니다

오늘의 요약

docker compose 명령 일곱 — 하려는 일로 찾기

하려는 일	docker compose 뒤에	기억할 점
최초 1회 초기화	up airflow-init	기동보다 먼저 한 번
기동	up -d	init 뒤에
상태 확인	ps	(healthy) 까지 1~2분
로그	logs -f airflow-scheduler	문제는 여기부터
멈춤 (데이터 유지)	stop	볼륨은 남는다
다시 시작	start	init 은 알아서 거친다
완전 삭제 (주의)	down --volumes	이력이 통째로 사라진다

CLI 는 af 함수(새 터미널마다 다시) — af dags list · af dags list-runs <dag> · af dags list-import-errors

구성 요소 복습

다섯 가지를 한 문장씩

구성 요소	비유	하는 일
API 서버	상황판	사람이 보는 화면(8080) · 워커의 보고 창구
DAG 프로세서	사서	dags/ 파일을 읽어 DAG 목록에 — 새 파일은 5분 주기
스케줄러	알람시계	시각이 되면 DAG 를 깨워 태스크를 큐에
워커	손	큐에서 태스크를 꺼내 실행 — 여럿 둘 수 있다
메타데이터 DB	일지	실행 이력의 정본 — 깨지면 통째로 사라진다

Redis 는 스케줄러와 워커 사이의 주문서 창구 · 트리거러는 기다리는 일 전담 — 일곱이 한 묶음으로 떠야 Airflow

오늘의 체크리스트

여덟 개를 스스로 확인해 보세요

1. 작업 사이에 순서가 생기면 cron 으로 부족한 이유를 설명할 수 있다
2. API 서버·스케줄러·DAG 프로세서·워커·DB 역할을 한 문장씩 말한다
3. compose 파일을 먼저 읽어 어떤 서비스가 뜨는지 확인했다
4. airflow-init 을 먼저 실행한 뒤 docker compose up -d 로 기동했다
5. docker compose ps 에서 일곱 서비스 Up (healthy) 를 확인했다
6. stop → start 로 데이터가 유지되는 것을 확인했다
7. Grid·Graph·Code·Logs 네 화면을 열고 상태 색을 읽을 수 있다
8. af 로 예제 DAG 를 unpause·trigger 하고 로그를 확인했다

자주 겪는 오류

증상으로 찾는 원인과 해결

증상	원인	해결
서비스들 계속 재시작 DB 오류 반복	airflow-init 을 건너뛰고 up -d	down --volumes 뒤 airflow-init 부터 순서대로
port is already allocated	다른 프로그램이 8080 을 쓰는 중	apiserver ports 를 "8081:8080" → :8081
exit code 137 전체가 매우 느림	Docker 메모리 부족 컨테이너 여럿 동시 기동	메모리 8GB 로 · 안 쓰는 컨테이너 stop
logs/ 에 Permission denied	.env 의 AIRFLOW_UID 없거나 불일치	AIRFLOW_UID=$(id -u) 로 다시 만들고 재기동

문제가 생기면 docker compose logs | grep -i "error\|exception" 부터

예상 질문

자주 나오는 질문 다섯 가지

1. **운영에서도 docker compose 로 쓰나요?**
   학습·소규모에는 쓴다 — 규모가 커지면 Kubernetes 나 관리형(MWAA·Cloud Composer)으로

2. **down 과 down --volumes 의 차이는?**
   앞은 컨테이너만 지우고 데이터는 남긴다 — 뒤는 메타데이터 DB 까지, 실행 이력이 전부 사라진다

3. **Airflow 말고 다른 도구는?**
   Dagster·Prefect·Mage — 개념(DAG·의존성·재시도)이 비슷해 하나를 알면 옮겨 가기 쉽다

4. **워커를 늘리려면?**
   compose 의 airflow-worker 를 여러 개로, 또는 --scale — 실습 노트북에서는 자원이 부족해진다

5. **왜 Airflow 3 인가요?**
   2 는 2.11 로 지원 종료 — DAG 쓰는 법은 거의 같고 화면 · import 경로가 바뀌었다

다음 차시 준비

Airflow 를 켜 둔 채로 마칩니다

1. **전 서비스 (healthy) 인지**
   cd ~/airflow-lab 에서 docker compose ps

2. **dags/ 폴더가 비어 있는지**
   ls -la dags/ — 8-2 DAG 파일을 여기에 쓴다

3. **af 함수 — 새 터미널마다 다시**
   af() { docker compose exec airflow-scheduler airflow "$@" < /dev/null; }

8-2 에서는

파일만 넣으면

읽어 간다

다음 회차 예고

8-2 · DAG 이해 및 유즈케이스 연구

01
이론 1H
DAG 핵심 개념
Task·Operator·스케줄

02
실습 1.5H
첫 DAG 작성
PythonOperator

03
실습 1H
유즈케이스 연구
태스크 분해안

04
정리 0.5H
DAG 작성 규칙
8-3 설계안 확정

준비: 8-1 의 Airflow 가 기동 상태 — docker compose ps 로 (healthy) 확인, 내려가 있으면 ~/airflow-lab 에서 docker compose up -d

8-1 · 정리·회고 0.5H 마무리

다음 회차는 8-2 DAG 이해

DAG·Task·Operator·의존성·스케줄 개념을 익히고, PythonOperator 로 첫 DAG 를 써서 dags/ 에 넣고 실행합니다

- ✓ **DAG·Task·Operator 의 관계를 그림으로**
- ✓ **PythonOperator 로 첫 DAG 작성**
- ✓ **의존성 연산자 >> 와 tasks test**
- ✓ **우리 파이프라인의 태스크 분해안**

준비할 것

Airflow 기동 상태 · 빈 dags/ 폴더